LiteLLM チュートリアル

— 100 以上の LLM を統一インターフェースで操る —

河合勝彦
名古屋市立大学大学院経済学研究科
kkawai@econ.nagoya-cu.ac.jp

2026 年 3 月 3 日

目次

1 はじめに 3
1.1 本チュートリアルについて 3
1.2 対応プロバイダー 3

2 環境構築 3
2.1 uv のインストール 3
2.2 API キーの設定 4
2.3 スクリプトの実行方法 4

3 基本的な使い方 4
3.1 基本的な補完呼び出し 4
3.2 複数プロバイダーの統一的な呼び出し 5

4 メッセージとプロンプト設計 6
4.1 システムプロンプトの活用 6
4.2 マルチターン会話 7

5 ストリーミング 9
5.1 同期ストリーミング 9
5.2 非同期ストリーミング 9

6 非同期処理と並列呼び出し 10
6.1 複数モデルの並列呼び出し 10

7 パラメータ制御 12
7.1 temperature パラメータ 12

7.2 主要パラメータ一覧 13

8 構造化出力 13
8.1 JSON モード 13
8.2 Pydantic モデルによる構造化出力 14

9 ツール呼び出し（Function Calling） 16

10 埋め込み（Embedding） 18

11 エラーハンドリング 19
11.1 例外マッピング 19
11.2 フォールバック 21

12 Router によるロードバランシング 22
12.1 ロードバランシング戦略 22

13 コールバックとオブザーバビリティ 24
13.1 カスタムコールバック 24
13.2 外部オブザーバビリティツールとの連携 26

14 コスト追跡 26

15 Proxy Server 27
15.1 インストールと起動 28
15.2 設定ファイル例 28
15.3 クライアントからの利用 28

16 vLLM との連携 29
16.1 vLLM の概要と役割分担 29
16.2 vLLM サーバーの起動 29
16.3 基本的な呼び出し 30
16.4 ストリーミング 31
16.5 埋め込み 32
16.6 ローカルとクラウドの混合構成 33
16.7 Proxy Server での vLLM 統合 35

17 スクリプト一覧 35

18 類似ツールとの比較 36
18.1 比較対象ツールの概要 36
18.2 機能比較 37
18.3 デプロイメントモデルと料金 37
18.4 用途に応じた選択指針 37

19 おわりに 39

1 はじめに

大規模言語モデル（LLM: Large Language Model）の利用が広がる中、OpenAI、Anthropic、Google など各社が独自の API を提供している。研究や開発においては複数のプロバイダーのモデルを比較・切り替えたい場面が多いが、プロバイダーごとに異なる API 仕様に対応するのは煩雑である。

LiteLLM[*1] は、100 以上の LLM API を **OpenAI 互換の統一インターフェース**で呼び出せる Python ライブラリである。主な特徴を以下に示す。

- **統一 API**: `completion()` 一つで OpenAI, Anthropic, Google Gemini, AWS Bedrock など多数のプロバイダーを呼び出せる
- **フォールバックとリトライ**: モデル障害時に自動で代替モデルに切り替え
- **ロードバランシング**: 複数デプロイメント間でリクエストを分散
- **コスト追跡**: API 呼び出しのコストを自動計算
- **ストリーミング**: 全プロバイダーで統一的なストリーミング応答
- **非同期対応**: `acompletion()` による非同期処理
- **Proxy Server**: OpenAI 互換の API ゲートウェイとしても動作

1.1 本チュートリアルについて

本チュートリアルでは、LiteLLM の基本的な使い方から応用的な機能まで、サンプルスクリプトを交えて解説する。各スクリプトは `uv` コマンドで直接実行できるように、PEP 723 のインラインメタデータ（`/// script` ブロック）を記述している。

1.2 対応プロバイダー

LiteLLM が対応する主なプロバイダーを表 1 に示す。

2 環境構築

2.1 uv のインストール

本チュートリアルでは、Python パッケージマネージャー uv[*2] を使用してスクリプトを実行する。uv は Rust 製の高速なパッケージマネージャーであり、依存関係の解決とスクリプト実行を一つのコマンドで行える。

```
# uv のインストール
curl -LsSf https://astral.sh/uv/install.sh | sh
```

[*1] https://docs.litellm.ai/
[*2] https://docs.astral.sh/uv/

表 1　LiteLLM の主な対応プロバイダー

プロバイダー	プレフィックス	代表的モデル
OpenAI	`openai/`	gpt-4o, gpt-4o-mini
Anthropic	`anthropic/`	claude-sonnet-4-5, claude-haiku-4-5
Google	`gemini/`	gemini-2.0-flash, gemini-1.5-pro
AWS Bedrock	`bedrock/`	anthropic.claude-3-sonnet 等
Azure OpenAI	`azure/`	デプロイメント名で指定
Mistral AI	`mistral/`	mistral-large-latest 等
Groq	`groq/`	llama-3.1-70b-versatile 等
Ollama	`ollama/`	llama3, phi3 等（ローカル）

2.2　API キーの設定

各プロバイダーの API キーを環境変数に設定する。

```
# 各プロバイダーの API キーを設定
export OPENAI_API_KEY="sk-..."
export ANTHROPIC_API_KEY="sk-ant-..."
export GEMINI_API_KEY="..."
```

注意

API キーはソースコードに直接記述せず、必ず環境変数や `.env` ファイルを通じて管理すること。

2.3　スクリプトの実行方法

本チュートリアルの各サンプルスクリプトは、PEP 723 のインラインメタデータを含んでおり、`uv run` で依存関係の自動インストールとともに実行できる。

```
# スクリプトの実行（依存関係は自動でインストールされる）
uv run scripts/01_basic_completion.py
```

3　基本的な使い方

3.1　基本的な補完呼び出し

最もシンプルな LiteLLM の使い方は、`completion()` 関数を呼び出すことである。OpenAI の Chat Completions API と同じフォーマットで、任意のプロバイダーのモデルを呼び出せる。

Listing 1　基本的な補完呼び出し（`scripts/01_basic_completion.py`）

```
# /// script
```

```
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""01_basic_completion.py - LiteLLMの基本的な補完呼び出し"""

import os
from litellm import completion

# 環境変数からAPIキーを読み込む（事前に設定しておくこと）
# export OPENAI_API_KEY="sk-..."

response = completion(
    model="openai/gpt-4o-mini",
    messages=[{"role": "user", "content":
        "Pythonの特徴を3つ挙げてください。"}],
)

print("=== レスポンス内容 ===")
print(response.choices[0].message.content)
print("\n=== トークン使用量 ===")
print(f"入力トークン: {response.usage.prompt_tokens}")
print(f"出力トークン: {response.usage.completion_tokens}")
print(f"合計トークン: {response.usage.total_tokens}")
print(f"\n=== モデル情報 ===")
print(f"モデル: {response.model}")
```

- model: **"プロバイダー/モデル名"** の形式で指定する
- messages: OpenAI 形式のメッセージリストを渡す
- レスポンスには choices（応答テキスト）と usage（トークン使用量）が含まれる

```
uv run scripts/01_basic_completion.py
```

3.2 複数プロバイダーの統一的な呼び出し

LiteLLM の最大の利点は、同じコードで異なるプロバイダーのモデルを呼び出せることである。モデル名のプレフィックスを変えるだけで、プロバイダーが自動的に切り替わる。

Listing 2 複数プロバイダーの呼び出し（scripts/02_multi_provider.py）

```
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""02_multi_provider.py - 複数プロバイダーの統一的な呼び出し"""
```

```python

import os
from litellm import completion

# 使用するモデルのリスト（プロバイダー/モデル名の形式）
models = [
    "openai/gpt-4o-mini",
    "anthropic/claude-haiku-4-5-20251001",
    "gemini/gemini-2.0-flash",
]

question = "機械学習とは何ですか？ 一文で説明してください。"

for model_name in models:
    try:
        response = completion(
            model=model_name,
            messages=[{"role": "user", "content": question}],
            max_tokens=200,
        )
        print(f"[{model_name}]")
        print(f"  回答: {response.choices[0].message.content}")
        print(f"  トークン: {response.usage.total_tokens}")
        print()
    except Exception as e:
        print(f"[{model_name}] エラー: {e}\n")
```

このスクリプトでは、OpenAI、Anthropic、Google Gemini の 3 つのプロバイダーに対して、同じ質問を同じインターフェースで送信している。これにより、モデル間の応答品質やトークン使用量を容易に比較できる。

```
uv run scripts/02_multi_provider.py
```

4 メッセージとプロンプト設計

4.1 システムプロンプトの活用

システムプロンプトを使うことで、モデルの振る舞い（ペルソナ、回答スタイル、専門分野など）を制御できる。

Listing 3 システムプロンプトの活用（scripts/03_system_prompt.py）

```python
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""03_system_prompt.py - システムプロンプトの活用"""
```

```python

from litellm import completion

# システムプロンプトでモデルの振る舞いを制御する
messages = [
    {
        "role": "system",
        "content": (
            "あなたは経済学の教授です。"
            "専門用語はなるべく平易な言葉で説明し、"
            "具体例を交えて回答してください。"
        ),
    },
    {
        "role": "user",
        "content": "インフレーションとは何ですか？ ",
    },
]

response = completion(
    model="openai/gpt-4o-mini",
    messages=messages,
    temperature=0.7,
    max_tokens=500,
)

print(response.choices[0].message.content)
```

messages リストの先頭に role: "system" のメッセージを配置することで、以降のユーザーメッセージに対するモデルの応答方針を設定できる。

```
uv run scripts/03_system_prompt.py
```

4.2 マルチターン会話

会話の文脈を保持するには、過去のやり取りをすべて messages リストに含める必要がある。以下のスクリプトでは、会話履歴を蓄積しながらマルチターンの対話を行う。

Listing 4 マルチターン会話（scripts/04_multi_turn.py）

```python
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""04_multi_turn.py - マルチターン会話"""

from litellm import completion

```

```python
# 会話履歴を保持するリスト
messages = [
    {
        "role": "system",
        "content": "あなたは親切なプログラミング講師です。",
    },
]


def chat(user_message: str) -> str:
    """ユーザーメッセージを送信し、応答を返す"""
    messages.append({"role": "user", "content": user_message})

    response = completion(
        model="openai/gpt-4o-mini",
        messages=messages,
        temperature=0.7,
    )

    assistant_message = response.choices[0].message.content
    messages.append({"role": "assistant", "content": assistant_message})
    return assistant_message


# マルチターン会話の実行
turns = [
    "Pythonでリストの基本操作を教えてください。",
    "リスト内包表記についても教えてください。",
    "では、それを使った実践的な例を示してください。",
]

for i, user_msg in enumerate(turns, 1):
    print(f"--- ターン {i} ---")
    print(f"ユーザー: {user_msg}")
    reply = chat(user_msg)
    print(f"アシスタント: {reply}\n")
```

LLM の API はステートレスであるため、文脈を維持するには呼び出し側でメッセージ履歴を管理する必要がある。各ターンで user と assistant のメッセージを交互に追加していくことで、自然な対話が実現できる。

```
uv run scripts/04_multi_turn.py
```

5 ストリーミング

5.1 同期ストリーミング

通常の API 呼び出しでは、モデルが応答を完全に生成し終えるまで結果が返らない。ストリーミングを使うと、生成されたテキストを逐次的に受け取ることができ、ユーザー体験が向上する。

Listing 5 同期ストリーミング（scripts/05_streaming.py）

```python
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""05_streaming.py - ストリーミング応答"""

from litellm import completion

print("=== 同期ストリーミング ===")
response = completion(
    model="openai/gpt-4o-mini",
    messages=[{"role": "user", "content":
        "日本の四季について短く説明してください。"}],
    stream=True,
)

# チャンクごとにリアルタイムで出力
for chunk in response:
    content = chunk.choices[0].delta.content
    if content:
        print(content, end="", flush=True)

print("\n")
```

stream=True を指定すると、レスポンスはイテレータとなり、for ループで各チャンクを順次処理できる。各チャンクの choices[0].delta.content にテキストの断片が格納されている。

```
uv run scripts/05_streaming.py
```

5.2 非同期ストリーミング

acompletion() を使うと、非同期でのストリーミングが可能になる。Web アプリケーションやイベント駆動型のシステムで有用である。

Listing 6 非同期ストリーミング（scripts/06_async_streaming.py）

```python
# /// script
# requires-python = ">=3.11"
```

```python
# dependencies = [
#     "litellm",
# ]
# ///
"""06_async_streaming.py - 非同期ストリーミング応答"""

import asyncio

from litellm import acompletion


async def async_stream():
    """非同期ストリーミングの例"""
    print("=== 非同期ストリーミング ===")
    response = await acompletion(
        model="openai/gpt-4o-mini",
        messages=[
            {"role": "user", "content":
                "統計学の基本概念を3つ説明してください。"}
        ],
        stream=True,
    )

    async for chunk in response:
        content = chunk.choices[0].delta.content
        if content:
            print(content, end="", flush=True)

    print()


asyncio.run(async_stream())
```

`async for` を使って非同期イテレーションを行う点が同期版との違いである。

```
uv run scripts/06_async_streaming.py
```

6 非同期処理と並列呼び出し

6.1 複数モデルの並列呼び出し

`asyncio.gather()` と `acompletion()` を組み合わせることで、複数のモデルに対して並列にリクエストを送信できる。これにより、順次実行に比べて大幅な時間短縮が期待できる。

Listing 7 非同期並列呼び出し（`scripts/07_async_parallel.py`）

```python
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
```

```python
# ]
# ///
"""07_async_parallel.py - 非同期による複数モデルの並列呼び出し"""

import asyncio
import time

from litellm import acompletion


async def call_model(model: str, question: str) -> dict:
    """単一モデルへの非同期呼び出し"""
    start = time.time()
    response = await acompletion(
        model=model,
        messages=[{"role": "user", "content": question}],
        max_tokens=200,
    )
    elapsed = time.time() - start
    return {
        "model": model,
        "response": response.choices[0].message.content,
        "time": elapsed,
        "tokens": response.usage.total_tokens,
    }


async def main():
    models = [
        "openai/gpt-4o-mini",
        "anthropic/claude-haiku-4-5-20251001",
        "gemini/gemini-2.0-flash",
    ]
    question =
        "データサイエンスで最も重要なスキルは何ですか？ 一文で答えてください。"

    print("=== 複数モデルへの並列リクエスト ===\n")
    start = time.time()

    # 全モデルに並列でリクエスト
    results = await asyncio.gather(
        *[call_model(m, question) for m in models],
        return_exceptions=True,
    )

    total_time = time.time() - start

    for result in results:
        if isinstance(result, Exception):
```

```
            print(f"エラー: {result}\n")
        else:
            print(f"[{result['model']}] ({result['time']:.2f}秒)")
            print(f"  回答: {result['response']}")
            print(f"  トークン数: {result['tokens']}\n")

    print(f"全体の実行時間: {total_time:.2f}秒")


asyncio.run(main())
```

ポイント

3 つのモデルに順次リクエストすると合計応答時間は各モデルの応答時間の和になるが、並列実行では最も遅いモデルの応答時間のみで済む。

```
uv run scripts/07_async_parallel.py
```

7 パラメータ制御

7.1 temperature パラメータ

temperature は応答のランダム性を制御するパラメータである。値が低いほど決定論的な応答となり、高いほど多様で創造的な応答が得られる。

- temperature=0.0: 最も決定論的。毎回ほぼ同じ応答
- temperature=0.5: やや保守的だが多少の変動あり
- temperature=1.0: デフォルト。バランスの取れたランダム性
- temperature=1.5: 高いランダム性。創造的だが不安定になりうる

Listing 8 temperature パラメータの効果（scripts/08_temperature.py）

```
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""08_temperature.py - temperatureパラメータの効果"""

from litellm import completion

prompt = "AIの未来について一文で予測してください。"
temperatures = [0.0, 0.5, 1.0, 1.5]

for temp in temperatures:
    print(f"=== temperature = {temp} ===")
```

```
    # 同じtemperatureで3回生成して、ばらつきを確認
    for i in range(3):
        response = completion(
            model="openai/gpt-4o-mini",
            messages=[{"role": "user", "content": prompt}],
            temperature=temp,
            max_tokens=100,
        )
        print(f"  [{i+1}] {response.choices[0].message.content}")
    print()
```

```
uv run scripts/08_temperature.py
```

7.2 主要パラメーター一覧

completion() 関数で使用可能な主要パラメータを表 2 にまとめる。

表 2 completion() の主要パラメータ

パラメータ	型	説明
model	str	モデル名（"**プロバイダー/モデル**" 形式）
messages	list	メッセージリスト
temperature	float	ランダム性（0.0–2.0）
top_p	float	Nucleus サンプリング（0.0–1.0）
max_tokens	int	最大出力トークン数
stop	list	生成停止文字列
stream	bool	ストリーミングの有効化
response_format	dict	応答形式（JSON モード等）
tools	list	ツール（関数）定義
tool_choice	str	ツール選択方法
timeout	float	タイムアウト（秒）
num_retries	int	リトライ回数

8 構造化出力

8.1 JSON モード

response_format パラメータに {"type": "json_object"} を指定することで、モデルに JSON 形式での応答を強制できる。データ抽出や構造化された情報の取得に有用である。

Listing 9 JSON モード（scripts/09_json_mode.py）

```
# /// script
# requires-python = ">=3.11"
```

```python
# dependencies = [
#     "litellm",
# ]
# ///
"""09_json_mode.py - JSONモードによる構造化出力"""

import json

from litellm import completion

# JSONモードを有効にする
response = completion(
    model="openai/gpt-4o-mini",
    response_format={"type": "json_object"},
    messages=[
        {
            "role": "system",
            "content": "あなたはJSON形式で回答するアシスタントです。",
        },
        {
            "role": "user",
            "content": (
                "以下の都市の情報をJSON形式で返してください: "
                "東京、大阪、名古屋。"
                "各都市について name, population（概算）, "
                "famous_for のフィールドを含めてください。"
            ),
        },
    ],
)

# レスポンスをパースして整形出力
raw = response.choices[0].message.content
data = json.loads(raw)
print(json.dumps(data, ensure_ascii=False, indent=2))
```

注意

JSON モードを使用する際は、システムプロンプトまたはユーザーメッセージで JSON 形式での応答を明示的に指示する必要がある。

```
uv run scripts/09_json_mode.py
```

8.2 Pydantic モデルによる構造化出力

Pydantic モデルを `response_format` に渡すことで、より厳密なスキーマに基づいた構造化出力が得られる。レスポンスを型安全なオブジェクトとして扱うことができる。

Listing 10　Pydantic による構造化出力（scripts/10_structured_output.py）

```python
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
#     "pydantic",
# ]
# ///
"""10_structured_output.py - Pydanticモデルによる構造化出力"""

import json

from pydantic import BaseModel
from litellm import completion


class BookReview(BaseModel):
    title: str
    author: str
    rating: float
    summary: str
    keywords: list[str]


response = completion(
    model="openai/gpt-4o-mini",
    messages=[
        {
            "role": "user",
            "content": (
                "夏目漱石の『坊っちゃん』の書評を作成してください。"
            ),
        },
    ],
    response_format=BookReview,
)

# Pydanticモデルとしてパース
review = BookReview.model_validate_json(
    response.choices[0].message.content
)

print(f"タイトル: {review.title}")
print(f"著者: {review.author}")
print(f"評価: {review.rating}/5.0")
print(f"要約: {review.summary}")
print(f"キーワード: {', '.join(review.keywords)}")
```

```
uv run scripts/10_structured_output.py
```

9 ツール呼び出し（Function Calling）

ツール呼び出し（Function Calling）は、モデルが外部の関数や API を呼び出す機能である。モデルが「どの関数をどのような引数で呼ぶべきか」を判断し、その結果を基に最終的な応答を生成する。

処理の流れは以下のとおりである。

1. ツール定義とユーザーメッセージをモデルに送信
2. モデルがツール呼び出し（関数名と引数）を返す
3. アプリケーション側で関数を実行し、結果をメッセージに追加
4. ツール結果を含めて再度モデルを呼び出し、最終応答を得る

Listing 11 ツール呼び出し（scripts/11_tool_calling.py）

```python
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""11_tool_calling.py - ツール呼び出し（Function Calling）"""

import json

from litellm import completion

# ツール（関数）の定義
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "指定された都市の現在の天気を取得する",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {
                        "type": "string",
                        "description": "都市名（例：東京）",
                    },
                    "unit": {
                        "type": "string",
                        "enum": ["celsius", "fahrenheit"],
                        "description": "温度の単位",
                    },
                },
                "required": ["city"],
```

```
34                 },
35             },
36         }
37 ]
38
39
40 # ダミーの天気関数
41 def get_weather(city: str, unit: str = "celsius") -> str:
42     """実際のAPIの代わりにダミーデータを返す"""
43     weather_data = {
44         "東京": {"temp": 22, "condition": "晴れ"},
45         "大阪": {"temp": 24, "condition": "曇り"},
46         "名古屋": {"temp": 23, "condition": "晴れ時々曇り"},
47     }
48     data = weather_data.get(city, {"temp": 20, "condition": "不明"})
49     return json.dumps(
50         {"city": city, "temperature": data["temp"],
51          "unit": unit, "condition": data["condition"]},
52         ensure_ascii=False,
53     )
54
55
56 # Step 1: ツール定義付きでモデルを呼び出す
57 messages = [
58     {"role": "user", "content": "東京と名古屋の天気を教えてください。"}
59 ]
60
61 response = completion(
62     model="openai/gpt-4o-mini",
63     messages=messages,
64     tools=tools,
65     tool_choice="auto",
66 )
67
68 response_message = response.choices[0].message
69 messages.append(response_message)
70
71 # Step 2: ツール呼び出しを実行
72 if response_message.tool_calls:
73     for tool_call in response_message.tool_calls:
74         func_name = tool_call.function.name
75         func_args = json.loads(tool_call.function.arguments)
76         print(f"ツール呼び出し: {func_name}({func_args})")
77
78         # 関数を実行
79         result = get_weather(**func_args)
80
81         # 結果をメッセージに追加
82         messages.append(
```

```
            {
                "tool_call_id": tool_call.id,
                "role": "tool",
                "name": func_name,
                "content": result,
            }
        )

    # Step 3: ツール結果を含めて再度モデルを呼び出す
    final_response = completion(
        model="openai/gpt-4o-mini",
        messages=messages,
    )

    print(f"\n最終回答:\n{final_response.choices[0].message.content}")
```

```
uv run scripts/11_tool_calling.py
```

10 埋め込み（Embedding）

テキスト埋め込み（Embedding）は、テキストを数値ベクトルに変換する機能であり、意味的な類似度計算や検索に利用される。LiteLLM は `embedding()` 関数で統一的な埋め込み生成インターフェースを提供する。

Listing 12 テキスト埋め込み（`scripts/12_embedding.py`）

```
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""12_embedding.py - テキスト埋め込み（Embedding）の生成"""

from litellm import embedding

# 埋め込みの生成
texts = [
    "機械学習は人工知能の一分野です。",
    "深層学習はニューラルネットワークを使用します。",
    "経済学は資源配分を研究する学問です。",
]

response = embedding(
    model="openai/text-embedding-3-small",
    input=texts,
)

```

```
# 結果の表示
for i, emb_data in enumerate(response.data):
    vec = emb_data["embedding"]
    print(f"テキスト{i+1}: 「{texts[i][:20]}...」")
    print(f"  次元数: {len(vec)}")
    print(f"  先頭5要素: {vec[:5]}")
    print()

# コサイン類似度の計算
import math


def cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = sum(x * y for x, y in zip(a, b))
    norm_a = math.sqrt(sum(x * x for x in a))
    norm_b = math.sqrt(sum(x * x for x in b))
    return dot / (norm_a * norm_b)


vecs = [d["embedding"] for d in response.data]
print("=== コサイン類似度 ===")
for i in range(len(texts)):
    for j in range(i + 1, len(texts)):
        sim = cosine_similarity(vecs[i], vecs[j])
        print(f"  テキスト{i+1} vs テキスト{j+1}: {sim:.4f}")
```

このスクリプトでは、3 つのテキストの埋め込みベクトルを生成し、コサイン類似度を計算している。意味的に近い「機械学習」と「深層学習」のテキストペアは、「経済学」との組み合わせよりも高い類似度を示すことが期待される。

```
uv run scripts/12_embedding.py
```

11 エラーハンドリング

11.1 例外マッピング

LiteLLM は各プロバイダー固有のエラーを OpenAI 互換の例外クラスにマッピングする。これにより、プロバイダーに依存しない統一的なエラーハンドリングが可能になる。

主な例外クラスを表 3 に示す。

Listing 13 エラーハンドリング（scripts/13_error_handling.py）

```
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""13_error_handling.py - 例外処理とエラーハンドリング"""
```

表 3　LiteLLM の主な例外クラス

HTTP ステータス	例外クラス	説明
400	BadRequestError	不正なリクエスト
400	ContextWindowExceededError	コンテキスト長超過
401	AuthenticationError	認証エラー
403	PermissionDeniedError	権限エラー
404	NotFoundError	リソースが見つからない
408	Timeout	タイムアウト
429	RateLimitError	レート制限
500	InternalServerError	サーバー内部エラー

```python

import litellm
from litellm import completion


def safe_completion(model: str, messages: list, **kwargs) -> str | None:
    """エラーハンドリング付きの補完関数"""
    try:
        response = completion(model=model, messages=messages, **kwargs)
        return response.choices[0].message.content

    except litellm.AuthenticationError as e:
        print(f"認証エラー: APIキーを確認してください - {e}")

    except litellm.RateLimitError as e:
        print(f"レート制限: しばらく待ってから再試行してください - {e}")

    except litellm.ContextWindowExceededError as e:
        print(f"コンテキスト長超過: 入力を短くしてください - {e}")

    except litellm.BadRequestError as e:
        print(f"不正なリクエスト: {e}")

    except litellm.APIConnectionError as e:
        print(f"API接続エラー: ネットワークを確認してください - {e}")

    except litellm.Timeout as e:
        print(f"タイムアウト: {e}")

    except Exception as e:
        print(f"予期しないエラー: {type(e).__name__}: {e}")

    return None

```

```python

# 正常なリクエスト
print("=== 正常なリクエスト ===")
result = safe_completion(
    model="openai/gpt-4o-mini",
    messages=[{"role": "user", "content": "こんにちは"}],
)
if result:
    print(f"応答: {result}\n")

# タイムアウトの例（非常に短いタイムアウト）
print("=== タイムアウトの例 ===")
result = safe_completion(
    model="openai/gpt-4o-mini",
    messages=[{"role": "user", "content": "長い文章を書いてください。"}],
    timeout=0.001,
)
```

```
uv run scripts/13_error_handling.py
```

11.2 フォールバック

あるモデルが利用不能になった場合に、自動的に別のモデルに切り替えるフォールバック機構を実装できる。

Listing 14 フォールバック（scripts/14_fallback.py）

```python
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""14_fallback.py - フォールバック（代替モデルへの切り替え）"""

from litellm import completion

# フォールバックリスト: メインモデルが失敗したら順に試行
fallback_models = [
    "openai/gpt-4o-mini",
    "anthropic/claude-haiku-4-5-20251001",
    "gemini/gemini-2.0-flash",
]


def completion_with_fallback(messages: list, **kwargs) -> str:
    """フォールバック付き補完関数"""
    errors = []
    for model in fallback_models:
```

```
        try:
            print(f"  試行中: {model}")
            response = completion(
                model=model, messages=messages, **kwargs
            )
            print(f"  成功: {model}")
            return response.choices[0].message.content
        except Exception as e:
            print(f"  失敗: {model} - {e}")
            errors.append((model, e))

    raise RuntimeError(
        f"全モデルが失敗しました: {errors}"
    )


# 実行例
messages = [
    {"role": "user", "content": "量子コンピュータとは何ですか? 一文で。"}
]
print("=== フォールバック実行 ===")
result = completion_with_fallback(messages, max_tokens=200)
print(f"\n回答: {result}")
```

```
uv run scripts/14_fallback.py
```

12 Router によるロードバランシング

Router クラスは、複数のモデルデプロイメント間でリクエストを分散させるロードバランサーである。リトライ、フォールバック、クールダウンなどの信頼性機能を内蔵している。

12.1 ロードバランシング戦略

Router が提供するロードバランシング戦略を表 4 に示す。

表 4 Router のロードバランシング戦略

戦略	説明
simple-shuffle	RPM/TPM の重みに基づくランダム分散（推奨）
latency-based-routing	応答時間が最も短いデプロイメントを選択
usage-based-routing	TPM 使用量が最も少ないデプロイメントを選択
least-busy	同時リクエスト数が最も少ないものを選択
cost-based-routing	最もコストの低いデプロイメントを選択

Listing 15 Router によるロードバランシング（scripts/15_router.py）

```python
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""15_router.py - Routerによるロードバランシング"""

import os
import asyncio

from litellm import Router

# 複数デプロイメントの設定
model_list = [
    {
        "model_name": "my-gpt",  # エイリアス名
        "litellm_params": {
            "model": "openai/gpt-4o-mini",
            "api_key": os.getenv("OPENAI_API_KEY"),
        },
    },
    {
        "model_name": "my-gpt",  # 同じエイリアスで別モデルも設定可能
        "litellm_params": {
            "model": "openai/gpt-4o-mini",
            "api_key": os.getenv("OPENAI_API_KEY"),
        },
    },
]

# Routerの初期化
router = Router(
    model_list=model_list,
    num_retries=3,            # リトライ回数
    fallbacks=[               # フォールバック設定
        {"my-gpt": ["my-gpt"]}
    ],
    routing_strategy="simple-shuffle",  # ロードバランシング戦略
)


async def main():
    # 複数リクエストを並列実行
    tasks = []
    for i in range(3):
        tasks.append(
            router.acompletion(
                model="my-gpt",
```

```
                messages=[
                    {
                        "role": "user",
                        "content": f"数字の{i+1}に関する豆知識を一つ。",
                    }
                ],
                max_tokens=100,
            )
        )

    responses = await asyncio.gather(*tasks, return_exceptions=True)

    for i, resp in enumerate(responses):
        if isinstance(resp, Exception):
            print(f"リクエスト{i+1}: エラー - {resp}")
        else:
            print(f"リクエスト{i+1}: {resp.choices[0].message.content}\n")


asyncio.run(main())
```

```
uv run scripts/15_router.py
```

13 コールバックとオブザーバビリティ

13.1 カスタムコールバック

LiteLLM のコールバック機構を使うと、API 呼び出しの前後にカスタムロジックを挿入できる。ログ記録、メトリクス収集、アラートなどに活用できる。

Listing 16 カスタムコールバック（scripts/16_custom_callback.py）

```
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""16_custom_callback.py - カスタムコールバックによるログ記録"""

import json
import time

import litellm
from litellm import completion
from litellm.integrations.custom_logger import CustomLogger


class SimpleLogger(CustomLogger):
```

```
    """API呼び出しのログを記録するカスタムロガー"""

    def __init__(self):
        self.logs = []

    def log_pre_api_call(self, model, messages, kwargs):
        print(f"[LOG] API呼び出し開始: model={model}")

    def log_success_event(self, kwargs, response_obj, start_time,
        end_time):
        elapsed = end_time - start_time
        entry = {
            "model": kwargs.get("model", "unknown"),
            "elapsed_seconds": elapsed.total_seconds(),
            "prompt_tokens": getattr(
                response_obj.usage, "prompt_tokens", 0
            ),
            "completion_tokens": getattr(
                response_obj.usage, "completion_tokens", 0
            ),
            "total_tokens": getattr(
                response_obj.usage, "total_tokens", 0
            ),
        }
        self.logs.append(entry)
        print(
            f"[LOG] 成功: {entry['model']} "
            f"({entry['elapsed_seconds']:.2f}秒, "
            f"{entry['total_tokens']}トークン)"
        )

    def log_failure_event(self, kwargs, response_obj, start_time,
        end_time):
        print(f"[LOG] 失敗: {kwargs.get('model', 'unknown')}")


# ロガーを登録
logger = SimpleLogger()
litellm.callbacks = [logger]

# 複数回のAPI呼び出し
models = ["openai/gpt-4o-mini", "openai/gpt-4o-mini"]
for model in models:
    response = completion(
        model=model,
        messages=[{"role": "user", "content": "1+1=?"}],
        max_tokens=50,
    )

```

```
# ログのサマリーを表示
print("\n=== ログサマリー ===")
print(json.dumps(logger.logs, indent=2, ensure_ascii=False))
```

CustomLogger を継承し、以下のフックメソッドをオーバーライドする。

- log_pre_api_call: API 呼び出し前に実行
- log_post_api_call: API 呼び出し後に実行
- log_success_event: 成功時に実行
- log_failure_event: 失敗時に実行

```
uv run scripts/16_custom_callback.py
```

13.2 外部オブザーバビリティツールとの連携

LiteLLM は以下のようなオブザーバビリティツールとの連携をサポートしている。

- **Langfuse**: LLM アプリケーションのトレーシングと分析
- **Lunary**: LLM モニタリングプラットフォーム
- **MLflow**: 実験管理とモデル追跡
- **Helicone**: API リクエストのログとメトリクス
- **OpenTelemetry**: 分散トレーシング標準

連携は litellm.success_callback にサービス名を指定するだけで有効化できる。

Listing 17 外部ツールとの連携（設定例）

```
import litellm

litellm.success_callback = ["langfuse", "lunary"]
litellm.failure_callback = ["sentry"]
```

14 コスト追跡

LiteLLM は API 呼び出しのコストを自動的に計算する機能を備えている。completion_cost() 関数にレスポンスオブジェクトを渡すことで、入出力トークン数に基づいたコストが算出される。

Listing 18 コスト追跡（scripts/17_cost_tracking.py）

```
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""17_cost_tracking.py - コスト追跡"""

import litellm
```

```python
from litellm import completion, completion_cost

# コスト追跡の例
models_and_prompts = [
    ("openai/gpt-4o-mini", "こんにちは"),
    ("openai/gpt-4o-mini", "Pythonの利点を5つ挙げてください。"),
    ("anthropic/claude-haiku-4-5-20251001", "機械学習とは何ですか？"),
]

total_cost = 0.0

for model, prompt in models_and_prompts:
    try:
        response = completion(
            model=model,
            messages=[{"role": "user", "content": prompt}],
            max_tokens=200,
        )

        # コスト計算
        cost = completion_cost(completion_response=response)
        total_cost += cost

        print(f"モデル: {model}")
        print(f"  入力トークン: {response.usage.prompt_tokens}")
        print(f"  出力トークン: {response.usage.completion_tokens}")
        print(f"  コスト: ${cost:.6f}")
        print()

    except Exception as e:
        print(f"モデル: {model} - エラー: {e}\n")

print(f"=== 合計コスト: ${total_cost:.6f} ===")
```

```
uv run scripts/17_cost_tracking.py
```

ポイント

コスト追跡はモデル間のコストパフォーマンス比較や、プロジェクトの予算管理に非常に有用である。Proxy Server を使えば、プロジェクト単位・ユーザー単位でのコスト管理も可能になる。

15 Proxy Server

LiteLLM Proxy Server は、LiteLLM を OpenAI 互換の API ゲートウェイとして動作させる機能である。チーム全体で共有する LLM ゲートウェイとして利用でき、以下の機能を提供する。

- 複数モデルへのルーティング
- 仮想 API キーの発行と管理
- レート制限
- プロジェクト単位のコスト追跡
- 管理ダッシュボード

15.1 インストールと起動

```
# Proxy 版のインストール
pip install 'litellm[proxy]'

# 設定ファイルで起動
litellm --config config.yaml
```

15.2 設定ファイル例

Listing 19 Proxy Server 設定ファイル（config.yaml）

```
model_list:
  - model_name: gpt-4o-mini
    litellm_params:
      model: openai/gpt-4o-mini
      api_key: os.environ/OPENAI_API_KEY

  - model_name: claude-haiku
    litellm_params:
      model: anthropic/claude-haiku-4-5-20251001
      api_key: os.environ/ANTHROPIC_API_KEY

  - model_name: gemini-flash
    litellm_params:
      model: gemini/gemini-2.0-flash
      api_key: os.environ/GEMINI_API_KEY

router_settings:
  routing_strategy: simple-shuffle
  num_retries: 3
```

15.3 クライアントからの利用

Proxy Server が起動すると、OpenAI 互換のエンドポイントが提供される。既存の OpenAI SDK を使ったコードをそのまま利用できる。

Listing 20 Proxy Server へのアクセス例

```
from openai import OpenAI
```

```python

# LiteLLM Proxy に接続
client = OpenAI(
    base_url="http://localhost:4000",
    api_key="sk-1234",  # Proxy の仮想キー
)

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Hello!"}],
)
print(response.choices[0].message.content)
```

16 vLLM との連携

vLLM[*3] は、LLM の推論を高速に実行するためのオープンソースライブラリである。PagedAttention や連続バッチング（Continuous Batching）などの最適化技術により、GPU メモリを効率的に活用しながら高いスループットを実現する。

LiteLLM は vLLM サーバーを **バックエンドの一つとして統合的に扱う**ことができる。これにより、ローカル GPU で動作するオープンソースモデルを、クラウド API と同じインターフェースで呼び出せる。

16.1 vLLM の概要と役割分担

LiteLLM と vLLM の役割を表 5 に整理する。

表 5 LiteLLM と vLLM の役割分担

	vLLM	**LiteLLM**
役割	LLM 推論エンジン	統一 API ゲートウェイ
動作場所	ローカル GPU サーバー	クライアント側 or Proxy サーバー
主な機能	モデルのロード、推論の高速化、OpenAI 互換 API の提供	複数バックエンドの統一呼び出し、ルーティング、コスト追跡
対象モデル	Hugging Face 上のオープンソースモデル	vLLM を含む 100 以上のプロバイダー

16.2 vLLM サーバーの起動

vLLM のインストールと、OpenAI 互換 API サーバーの起動方法を以下に示す。

[*3] https://docs.vllm.ai/

```
# vLLM のインストール
pip install vllm

# チャットモデルのサーバー起動（例: Llama 3.1 8B）
vllm serve meta-llama/Llama-3.1-8B-Instruct --port 8000

# 埋め込みモデルのサーバー起動（別ポートで）
vllm serve intfloat/multilingual-e5-large-instruct --port 8001
```

GPU 要件

vLLM には CUDA 対応 GPU（Compute Capability ≥ 7.0）が必要である。代表的な対応 GPU は V100、T4、RTX 20xx 以降、A100、L4、H100 などである。`--gpu-memory-utilization` オプションで VRAM 使用率を調整できる（デフォルト: 0.9）。

16.3 基本的な呼び出し

vLLM サーバーに対して LiteLLM から呼び出すには、モデル名に `hosted_vllm/` プレフィックスを付け、`api_base` にサーバーの URL を指定する。

Listing 21 vLLM サーバー経由の補完呼び出し（`scripts/18_vllm_completion.py`）

```python
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""18_vllm_completion.py - vLLMサーバー経由の補完呼び出し

事前に vLLM サーバーを起動しておくこと:
  vllm serve meta-llama/Llama-3.1-8B-Instruct --port 8000
"""

from litellm import completion

# vLLM サーバーのベースURL（デフォルト: http://localhost:8000）
VLLM_API_BASE = "http://localhost:8000"

response = completion(
    model="hosted_vllm/meta-llama/Llama-3.1-8B-Instruct",
    messages=[
        {"role": "system", "content": "あなたは親切なアシスタントです。"},
        {"role": "user", "content":
            "Pythonのリスト内包表記について簡潔に説明してください。"},
    ],
    api_base=VLLM_API_BASE,
    temperature=0.7,
```

```
    max_tokens=300,
)

print("=== vLLM レスポンス ===")
print(response.choices[0].message.content)
print(f"\nモデル: {response.model}")
print(f"入力トークン: {response.usage.prompt_tokens}")
print(f"出力トークン: {response.usage.completion_tokens}")
```

クラウド API の呼び出しと同じ completion() 関数を使っている点に注目されたい。model と api_base を変えるだけで、ローカルの vLLM サーバーにリクエストが送られる。

```
uv run scripts/18_vllm_completion.py
```

16.4 ストリーミング

vLLM サーバーからのストリーミング応答も、クラウド API と同様に stream=True で利用できる。

Listing 22 vLLM ストリーミング（scripts/19_vllm_streaming.py）

```
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""19_vllm_streaming.py - vLLMサーバーからのストリーミング応答

事前に vLLM サーバーを起動しておくこと:
  vllm serve meta-llama/Llama-3.1-8B-Instruct --port 8000
"""

from litellm import completion

VLLM_API_BASE = "http://localhost:8000"

print("=== vLLM ストリーミング ===")
response = completion(
    model="hosted_vllm/meta-llama/Llama-3.1-8B-Instruct",
    messages=[
        {"role": "user", "content":
            "日本の経済史を3つの時代に分けて簡潔に説明してください。"},
    ],
    api_base=VLLM_API_BASE,
    stream=True,
    max_tokens=500,
)

for chunk in response:
```

```
    content = chunk.choices[0].delta.content
    if content:
        print(content, end="", flush=True)

print()
```

```
uv run scripts/19_vllm_streaming.py
```

16.5 埋め込み

vLLM は埋め込みモデルの推論もサポートしている。LiteLLM の `embedding()` 関数から、vLLM サーバー上の埋め込みモデルを呼び出せる。

Listing 23 vLLM による埋め込み生成（`scripts/20_vllm_embedding.py`）

```
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""20_vllm_embedding.py - vLLMサーバーによる埋め込み生成

事前に埋め込みモデルで vLLM サーバーを起動しておくこと:
  vllm serve intfloat/multilingual-e5-large-instruct --port 8001
"""

from litellm import embedding

VLLM_API_BASE = "http://localhost:8001"

texts = [
    "自然言語処理は人工知能の一分野です。",
    "テキストマイニングで文書を分析します。",
    "株式市場は経済の動向を反映します。",
]

response = embedding(
    model="hosted_vllm/intfloat/multilingual-e5-large-instruct",
    input=texts,
    api_base=VLLM_API_BASE,
)

for i, emb_data in enumerate(response.data):
    vec = emb_data["embedding"]
    print(f"テキスト{i+1}: 「{texts[i][:20]}...」")
    print(f"  次元数: {len(vec)}")
    print(f"  先頭5要素: {vec[:5]}")
    print()
```

```
uv run scripts/20_vllm_embedding.py
```

16.6 ローカルとクラウドの混合構成

LiteLLM の Router を使うと、ローカルの vLLM サーバーをプライマリとして使用し、サーバーが停止している場合やエラー発生時にクラウド API へ自動フォールバックする構成を実現できる。

Listing 24 vLLM とクラウド API の混合利用（scripts/21_vllm_multi_backend.py）

```
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "litellm",
# ]
# ///
"""21_vllm_multi_backend.py - vLLMとクラウドAPIの混合利用

ローカルの vLLM サーバーとクラウド API を同じコードで切り替える例。
Router を使えば、ローカルモデルを優先しつつクラウドにフォールバックできる。

事前に vLLM サーバーを起動しておくこと:
  vllm serve meta-llama/Llama-3.1-8B-Instruct --port 8000
"""

import asyncio
import os
import time

from litellm import Router

# ローカル vLLM + クラウド API の混合構成
model_list = [
    {
        "model_name": "my-llm",
        "litellm_params": {
            "model": "hosted_vllm/meta-llama/Llama-3.1-8B-Instruct",
            "api_base": "http://localhost:8000",
            "rpm": 100,   # ローカルなので高い RPM を設定
        },
        "model_info": {
            "description": "ローカル vLLM サーバー（優先）",
        },
    },
    {
        "model_name": "my-llm",
        "litellm_params": {
            "model": "openai/gpt-4o-mini",
```

```
            "api_key": os.getenv("OPENAI_API_KEY"),
            "rpm": 30,
        },
        "model_info": {
            "description": "クラウド API（フォールバック）",
        },
    },
]

router = Router(
    model_list=model_list,
    fallbacks=[{"my-llm": ["my-llm"]}],
    num_retries=2,
    routing_strategy="simple-shuffle",
)


async def main():
    question = "回帰分析とは何ですか？ 一文で簡潔に答えてください。"

    print("=== vLLM + クラウド API 混合構成 ===\n")
    start = time.time()

    # Router がローカル vLLM を優先し、失敗時にクラウドへフォールバック
    response = await router.acompletion(
        model="my-llm",
        messages=[{"role": "user", "content": question}],
        max_tokens=200,
    )

    elapsed = time.time() - start
    print(f"使用モデル: {response.model}")
    print(f"回答: {response.choices[0].message.content}")
    print(f"応答時間: {elapsed:.2f}秒")


asyncio.run(main())
```

この構成には以下の利点がある。

- **コスト削減**: ローカル GPU を活用することで、API 呼び出しコストを大幅に削減できる
- **データプライバシー**: 機密データをローカルで処理し、外部に送信しない
- **高可用性**: ローカルサーバー障害時にクラウドへ自動切り替え
- **レイテンシ最適化**: ローカル推論によるネットワーク遅延の削減

```
uv run scripts/21_vllm_multi_backend.py
```

16.7 Proxy Server での vLLM 統合

LiteLLM Proxy Server の設定ファイルに vLLM サーバーを追加することで、チーム全体でローカルモデルとクラウドモデルを透過的に利用できる。

Listing 25 vLLM を含む Proxy Server 設定（config.yaml）

```yaml
model_list:
  # ローカル vLLM サーバー
  - model_name: llama-local
    litellm_params:
      model: hosted_vllm/meta-llama/Llama-3.1-8B-Instruct
      api_base: http://localhost:8000
      rpm: 100

  # クラウド API（フォールバック先）
  - model_name: llama-local
    litellm_params:
      model: openai/gpt-4o-mini
      api_key: os.environ/OPENAI_API_KEY
      rpm: 30

  # 埋め込みモデル（vLLM）
  - model_name: embedding-local
    litellm_params:
      model: hosted_vllm/intfloat/multilingual-e5-large-instruct
      api_base: http://localhost:8001

router_settings:
  routing_strategy: simple-shuffle
  num_retries: 2
```

ポイント

vLLM と LiteLLM の組み合わせは、「ローカル GPU の計算資源を最大限活用しつつ、クラウド API の利便性と信頼性も確保する」ハイブリッド構成を低コストで実現する。研究室のオンプレミス GPU サーバーに vLLM を配置し、LiteLLM Proxy で一元管理するアーキテクチャは、学術研究の現場でも有効な構成である。

17 スクリプト一覧

本チュートリアルで使用したサンプルスクリプトの一覧を表 6 に示す。

表 6: サンプルスクリプト一覧

ファイル名	内容	節
01_basic_completion.py	基本的な補完呼び出し	3.1
02_multi_provider.py	複数プロバイダーの呼び出し	3.2
03_system_prompt.py	システムプロンプトの活用	4.1
04_multi_turn.py	マルチターン会話	4.2
05_streaming.py	同期ストリーミング	5.1
06_async_streaming.py	非同期ストリーミング	5.2
07_async_parallel.py	非同期並列呼び出し	6.1
08_temperature.py	temperature パラメータ	7.1
09_json_mode.py	JSON モード	8.1
10_structured_output.py	Pydantic による構造化出力	8.2
11_tool_calling.py	ツール呼び出し	9
12_embedding.py	テキスト埋め込み	10
13_error_handling.py	エラーハンドリング	11.1
14_fallback.py	フォールバック	11.2
15_router.py	Router	12
16_custom_callback.py	カスタムコールバック	13.1
17_cost_tracking.py	コスト追跡	14
18_vllm_completion.py	vLLM 基本呼び出し	16.3
19_vllm_streaming.py	vLLM ストリーミング	16.4
20_vllm_embedding.py	vLLM 埋め込み生成	16.5
21_vllm_multi_backend.py	vLLM+ クラウド混合構成	16.6

すべてのスクリプトは以下のコマンドで実行できる。

```
uv run scripts/<ファイル名>.py
```

18 類似ツールとの比較

LiteLLM と同様に、複数の LLM プロバイダーを統一的に扱うツールやサービスが存在する。本節では代表的な類似ツールを取り上げ、LiteLLM との違いを整理する。

18.1 比較対象ツールの概要

18.1.1 OpenRouter

OpenRouter[*4] は、400 以上の LLM モデルに単一の API エンドポイントからアクセスできる **マネージド SaaS 型**の LLM ゲートウェイである。インフラの構築やサーバーの運用が不要で、API

[*4] https://openrouter.ai/

キーを取得するだけですぐに利用を開始できる。料金は各プロバイダーの実費にプラットフォーム手数料（5–5.5%）を加えた従量課金制である。自動モデル選択やプロバイダーレベルのフォールバックルーティングも備えるが、カスタムルーティングロジックの柔軟性は LiteLLM に劣る。

18.1.2 aisuite

aisuite[*5] は、Andrew Ng らが開発したオープンソースの Python ライブラリである。LiteLLM と同様に **"プロバイダー: モデル"** 形式でモデルを指定し、OpenAI 互換のインターフェースで呼び出せる。最大の特徴は**軽量さ**であり、各プロバイダーの SDK を薄くラップするシンプルな設計となっている。Python 関数を直接ツールとして渡せるエージェント機能も備えるが、Proxy Server やロードバランシングなどの運用機能は持たない。

18.1.3 LangChain

LangChain[*6] は、LLM を活用したアプリケーション構築のための包括的フレームワークである。チャットモデルの呼び出しだけでなく、RAG（検索拡張生成）、エージェント、メモリ管理、ツール連携など、複雑なワークフローの構築を支援する。LLM ゲートウェイとしての機能は LiteLLM ほど特化していないが、アプリケーションレイヤーの機能が非常に豊富である。

18.1.4 Portkey

Portkey[*7] は、LLM アプリケーション向けのオブザーバビリティおよびゲートウェイプラットフォームである。マルチプロバイダーのルーティング、フォールバック、キャッシュ、レート制限、トークン使用量追跡などの機能を提供し、OpenAI 互換の Proxy として動作する。クラウド版とセルフホスト版の両方が利用可能で、エンタープライズ向けのガバナンス機能が充実している。

18.2 機能比較

主要な機能の比較を表 7 に示す。

18.3 デプロイメントモデルと料金

各ツールのデプロイメントモデルと料金体系を表 8 に示す。

18.4 用途に応じた選択指針

- **手軽に複数モデルを比較したい**: aisuite が最も軽量で導入が容易。研究段階でのプロトタイピングに適する
- **セルフホストで本格的な LLM ゲートウェイを運用したい**: LiteLLM が最適。ロードバランシング、フォールバック、コスト追跡、仮想キー管理など、本番運用に必要な機能が揃っている

[*5] https://github.com/andrewyng/aisuite
[*6] https://www.langchain.com/
[*7] https://portkey.ai/

表 7　LiteLLM と類似ツールの機能比較

機能	LiteLLM	OpenRouter	aisuite	LangChain	Portkey
統一 API					
対応モデル数	100+	400+	20+	50+	100+
Proxy Server		—	—	—	
ロードバランシング		限定的	—	—	
フォールバック			—		
コスト追跡			—	—	
ストリーミング					
非同期対応		—			
ツール呼び出し					
埋め込み			—		
RAG・エージェント	—	—	限定的		—
オブザーバビリティ		限定的	—		

表 8　デプロイメントモデルと料金の比較

ツール	デプロイメント	料金
LiteLLM	セルフホスト（OSS）/ Enterprise	OSS 版は無料。Enterprise 版は年額約 $30,000。セルフホストのインフラ費用が別途必要
OpenRouter	マネージド SaaS	プロバイダー実費 + プラットフォーム手数料 5–5.5%。インフラ管理不要
aisuite	ライブラリ（OSS）	完全無料（MIT ライセンス）。各プロバイダーの API 料金のみ
LangChain	ライブラリ（OSS）/ LangSmith	OSS 版は無料。LangSmith（監視・評価）は有料プランあり
Portkey	クラウド / セルフホスト	無料プランあり。有料プランは従量課金制

- **インフラ管理なしで多数のモデルにアクセスしたい**: OpenRouter が適する。SaaS 型のため運用負荷がゼロで、400 以上のモデルに即座にアクセスできる
- **RAG やエージェントなど複雑なワークフローを構築したい**: LangChain が包括的な機能を提供する。LLM ゲートウェイとしては LiteLLM と組み合わせて使うことも可能
- **エンタープライズ向けのガバナンスと監視が必要**: Portkey がオブザーバビリティとガバナンス機能に優れる

LiteLLM の位置づけ

LiteLLM は「セルフホスト可能で機能が豊富な OSS の LLM ゲートウェイ」として、aisuite の軽量さと OpenRouter のマネージドサービスの中間に位置する。特に、データの主権を維持しつつ高度なルーティングやコスト管理が必要な研究・開発環境において、強力な選択肢となる。

19 おわりに

本チュートリアルでは、LiteLLM の基本的な使い方から、ストリーミング、非同期処理、構造化出力、ツール呼び出し、埋め込み、エラーハンドリング、ロードバランシング、コールバック、コスト追跡、Proxy Server まで幅広く解説した。

LiteLLM を活用することで、以下の利点が得られる。

- **ベンダーロックインの回避**: プロバイダーの切り替えがモデル名の変更だけで完了する
- **比較実験の効率化**: 同じコードで複数モデルの性能を比較できる
- **プロダクション対応**: Router や Proxy Server により、本番環境での運用にも対応できる
- **コスト管理**: 自動的なコスト計算により、研究予算の管理が容易になる

LLM の進化は急速であり、新しいモデルやプロバイダーが次々と登場している。LiteLLM のような抽象化レイヤーを活用することで、技術の進歩に柔軟に対応しながら、本質的な研究や開発に集中できるようになる。

詳細な API リファレンスや最新の対応モデルについては、公式ドキュメント[*8]を参照されたい。

[*8] https://docs.litellm.ai/